# Assignment 1 — Digital Loan Processing System

## Problem Statement

A financial institution wants to design a **Digital Loan Processing System** to manage different types of loans issued to customers.

Each loan has common attributes such as:

- loan ID
- borrower name
- principal amount
- interest rate

However, the bank supports different types of loans such as:

- Home Loan
- Car Loan
- Education Loan

Each loan type may calculate repayment information differently.

The bank also wants the system to verify whether a borrower is eligible to apply for a loan based on certain checks. These eligibility checks must be designed in a flexible way so that different loan types can apply their own rules.

The system must also ensure that invalid loan values (such as negative loan amount or invalid interest rate) do not enter the system.

In addition, the bank wants to maintain a global configuration that loads when the system starts.

# Tasks

Design a Java program that:

1. Uses an **abstract base class** representing a general loan.
2. Uses **inheritance** to create specific loan types.
3. Uses **method overriding** so that each loan type can implement its own repayment or calculation logic.
4. Uses an **interface** to represent loan eligibility checking.
5. Demonstrates **polymorphism** while processing multiple loan objects through a common reference.
6. Uses a **static block** to initialize system-level configuration when the program starts.
7. Demonstrates **constructor calling order** when objects are created.
8. Uses **exception handling** to prevent invalid loan values from being accepted.
9. Defines at least one **custom exception** for invalid loan input.

# Expected Program Behavior

The system should:

- create multiple loan objects
- store them in a collection or array
- process them through a common workflow
- show that each loan type behaves according to its own implementation
- gracefully handle invalid loan inputs using custom exceptions

# Assignment 2 —Hospital Appointment and Treatment System

## Problem Statement

A hospital wants to build a **Hospital Appointment and Treatment System** to manage different types of medical services provided to patients.

All medical services share some common information such as:

- service ID
- patient name
- consultation fee

However, the hospital offers multiple service categories such as:

- General Consultation
- Surgery
- Diagnostic Test

Each category follows different procedures and fee calculations.

The hospital also requires that each service verify whether it meets operational requirements before being scheduled.

The system must prevent invalid service details (such as negative fees or invalid IDs) from being recorded.

The hospital system also requires certain hospital configuration data to be loaded automatically when the system starts.

# Tasks

Design a Java program that:

1. Defines an **abstract class** representing a general hospital service.
2. Creates specialized service classes using **inheritance**.
3. Uses **method overriding** to implement service-specific behavior.
4. Defines an **interface** for service validation or eligibility checking.
5. Demonstrates **polymorphism** when processing different service types through a common reference.
6. Uses a **static block** to initialize hospital configuration when the system loads.
7. Demonstrates **constructor calling sequence** during object creation.
8. Uses **exception handling** to handle invalid service data.
9. Defines a **custom exception class** to represent invalid service configuration.

# Expected Program Behavior

The system should:

- create multiple service records
- store them in an array or collection
- process them uniformly using polymorphism
- show specialized behavior for each service type
- handle invalid service data using custom exceptions

# Assignment 3 — Online Order Fulfillment System

## Problem Statement

An e-commerce company wants to implement a simplified **Online Order Fulfillment System** to process different types of product orders.

Each order contains common information such as:

- order ID
- customer name
- order amount

However, orders may belong to different categories such as:

- Standard Order
- Express Order
- International Order

Each category follows different processing rules.

The company also wants to ensure that all orders go through a verification process before they are accepted.

The system must ensure that invalid orders (such as negative order amount or missing details) are not processed.

Additionally, the system should load certain application configuration values when the program starts.

# Tasks

Design a Java program that:

1. Creates an **abstract class** representing a general order.
2. Uses **inheritance** to define specialized order types.
3. Uses **method overriding** so each order type can implement its own processing logic.
4. Defines an **interface** representing order verification.
5. Demonstrates **polymorphism** by processing multiple orders through a common reference.
6. Uses a **static block** to initialize system configuration.
7. Shows **constructor execution order** during object creation.
8. Uses **exception handling** for invalid order inputs.
9. Implements a **custom exception** for invalid order scenarios.

# Expected Program Behavior

The system should:

- create several order objects
- process them through a common order handling workflow
- demonstrate polymorphic behavior for different order types
- properly handle invalid order inputs using custom exceptions